

REQ1 – Limveld

A. Teleportation Mechanism Implementation

Design 1: Embedding Teleport Logic Directly in Actor Class

Pros	Cons
Simpler class hierarchy with no extra action subclasses	Actor handles both core and teleport logic
Direct access to Actor internals for teleport logic, enabling optimizations	Teleport logic tied tightly to Actor, limiting reuse in other contexts
	Any change to teleport requires Actor modification, increasing risk of bugs
	Code becomes cluttered as teleport logic grows

Design 2: Independent TeleportAction Class

Pros	Cons
Clear separation of concerns: teleport behavior isolated in TeleportAction class	Requires understanding of Action abstraction, adding conceptual overhead
Looser coupling allows reuse across different Actor types or map elements	Actors or map elements must explicitly register TeleportAction for functionality
Easier to extend or override teleport behavior via subclassing or composition	

B. Behaviour Selection Strategy

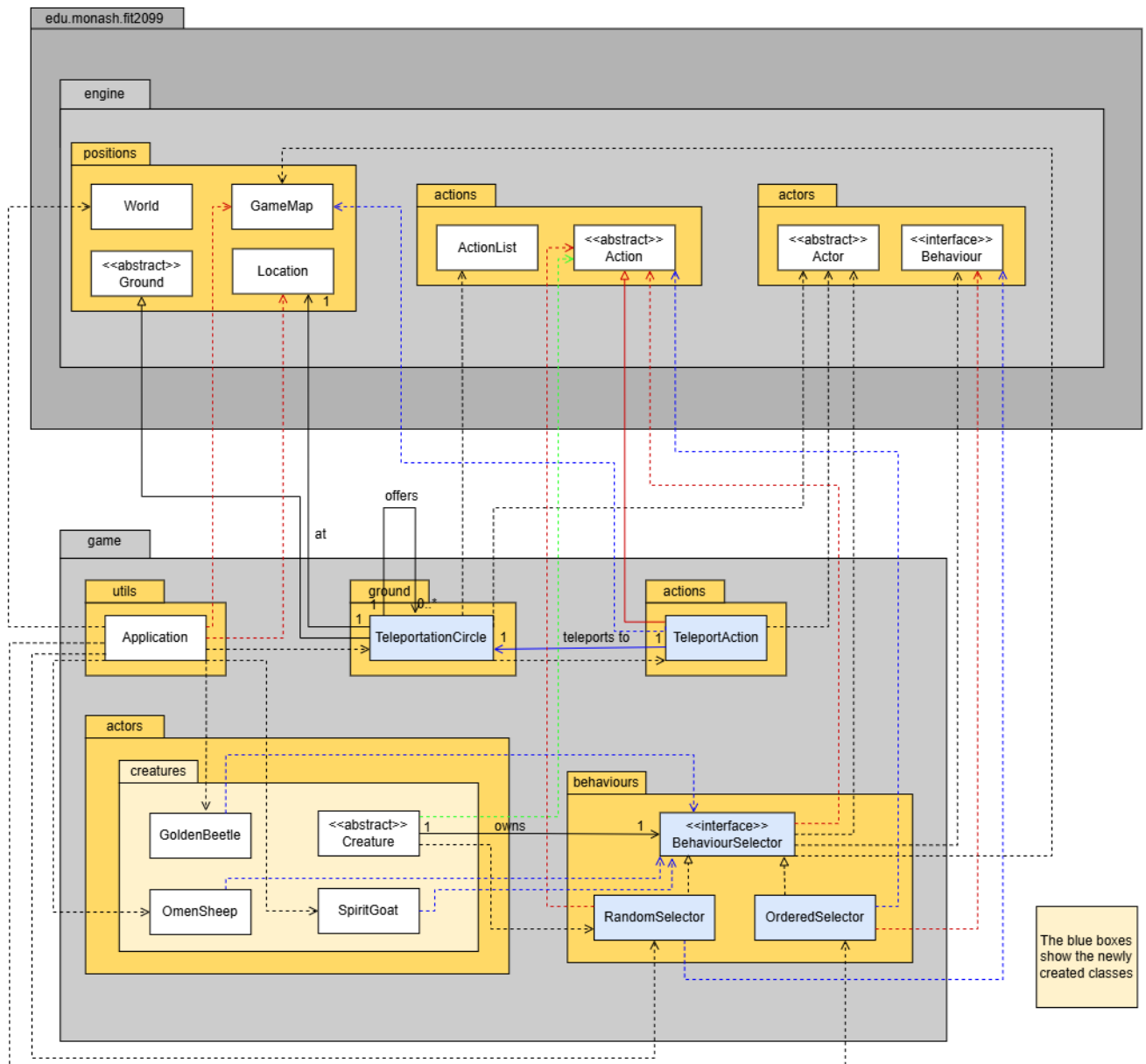
Design 1: Hardcoding Selection Logic in Actor

Pros	Cons
All decision logic centralized, making it quick to find and modify	Changes to logic require editing Actor code
No extra interfaces or design patterns to learn	High coupling reduces code reuse across different actor types

Design 2: BehaviourSelector Interface

Pros	Cons
Encapsulates decision logic, improving modularity and separation of concerns	More classes and interfaces increase codebase size
Supports easy extension by adding new selectors without modifying Actor	Possible over-abstraction for very simple actors
Enables reuse of selectors across different Actors or NPC types	

UML REQ1



The selected approaches are as follows: for the **Teleportation Mechanism**, Design 2 was chosen, which uses an independent TeleportAction. The **Behaviour Selection Strategy** adopts Design 2, utilizing a BehaviourSelector interface.

Using protected access modifier for both behaviours and selector in the Creature class allows subclasses like OmenSheep and SpiritGoat to directly configure and modify their behavior logic. This supports flexible customization without exposing internals to unrelated classes, promoting good encapsulation while enabling extensible and readable subclass design without unnecessary boilerplate methods.

Apart from that, `public static final int EGG_TIMER_THRESHOLD` is declared `public` so that external classes—like the `Application` class—can access it. This allows the game to reference

the Omen Sheep's default egg timer value (e.g., for display, configuration, or debugging) without instantiating the class. It promotes reusability and consistency.

This design adheres closely to the **Single Responsibility Principle SRP**. TeleportAction is solely responsible for the teleport logic, allowing actors to remain focused on their own state and executing actions. Meanwhile, the TeleportationCircle manages only a list of destination teleportation circles and generates corresponding TeleportAction instances. The BehaviourSelector interface and its implementations are dedicated exclusively to deciding “which behaviour to pick,” without mixing behaviour execution logic.

The **Open/Closed Principle OCP** is maintained by enabling new teleport variants, such as a “TeleportWithParticleEffect,” to be added through subclassing or wrapping TeleportAction. This means existing classes remain untouched. Similarly, new behaviour-selection strategies like WeightedSelector or CompositeSelector can be introduced simply by implementing the BehaviourSelector interface, without modifying existing actors or selectors.

The design follows the **Liskov Substitution Principle LSP** since any class implementing BehaviourSelector can seamlessly replace another in an actor’s constructor. Likewise, any subclass of TeleportAction can substitute the base class without breaking existing teleportation calls, as they adhere to the same Action contract.

In terms of the **Interface Segregation Principle ISP**, BehaviourSelector contains only the method selectAction(...), so actors aren’t forced to depend on unused methods. If more complex behaviour selection is required in the future (e.g., bulk evaluation), separate interfaces can be introduced without bloating the current one.

Regarding the **Dependency Inversion Principle DIP**, actors depend on the abstraction BehaviourSelector rather than any concrete implementations like RandomSelector or OrderedSelector. Similarly, map or ground classes depend on the Action interface for teleportation, rather than any specific teleport action class, which enhances flexibility and testability.

The design also embraces **DRY (Don’t Repeat Yourself)** principles by centralizing all teleport logic inside the TeleportAction class. If teleport behaviour changes (for example, by adding a mana cost), only that class needs modification. Behaviour-selection logic is not duplicated across actors since all actors reuse the same selector implementations.

Connascence Minimization is achieved by ensuring that TeleportAction and TeleportationCircle depend only on Location and Action, without relying on the internals of Actor or GameMap. Similarly, BehaviourSelector implementations depend only on the behaviour contract, not on specific behaviour details. Actors only need to know that teleport

or selection strategies conform to their abstract interfaces, without knowing their implementation details.

In this design, the primary types of connascence present are **Connascence of Name (CoN)** and **Connascence of Type (CoT)**, as components like `TeleportAction`, `TeleportationCircle`, and `BehaviourSelector` must consistently reference and use the same class and interface names (e.g., `Location`, `Action`, `BehaviourSelector`) and rely on their specific type signatures. Additionally, there is **Connascence of Identity (CoI)** between `Actor` and implementations of `BehaviourSelector`, since all parties must conform to the same method contract (`selectAction(...)`). More fragile forms like **Connascence of Position (CoP)** and **Connascence of Meaning (CoM)** are avoided due to the clean separation of concerns and use of well-defined abstractions, which helps minimize tight coupling and enhances maintainability.

Future Maintainability & Extensibility are well supported. New teleport variants, such as a portal that requires a key or has a cooldown, can be created by subclassing or decorating `TeleportAction`. Additional behaviour selectors can be implemented, such as a `WeightedSelector` that chooses based on behaviour weights, or a `ContextualSelector` that factors in actor health or distance. Actors can easily switch strategies by injecting different selectors.

For **Concurrency**, if teleport targets might be modified at runtime by multiple threads (due to dynamic world changes), the `List<TeleportationCircle>` can be replaced with a thread-safe collection or synchronized during modification.

By combining an independent `TeleportAction`, a simple yet flexible target list, and a strategy-based behaviour-selection interface (with both random and ordered implementations), this design ensures clear separation of concerns, strong adherence to SOLID principles, minimal coupling, and maximum extensibility for future game features.

Sequence Diagram REQ1

